

مستند طراحی و معماری مدل‌های تشخیص ناهنجاری

LSTM-AE، Isolation Forest، گیت حمله شناخته‌شده و Random Forest دومرحله‌ای

نسخه	۱/۰ — بر پایه پیاده‌سازی فعلی
دامنه	سامانه تشخیص ناهنجاری شبکه مبتنی بر PCAP
مخاطب	تیم فنی، ارزیاب معماری و بهره‌بردار سامانه
تاریخ	۱۴۰۵/۰۶/۱۸

این گزارش از روی قراردادهای و کدهای فعلی پروژه تولید شده است. نمودارهای داخل سند، مسیرهای پیاده‌سازی شده را نشان می‌دهند و صرفاً تصویر مفهومی نیستند.

۱. مسئله، اهداف و اصول طراحی مدل

صورت مسئله

سامانه باید از featureهای استخراج شده از PCAP ابتدا تشخیص دهد آیا یک پکت یا رفتار وابسته به flow ناهنجار است و سپس، فقط در صورت عبور از Stage ۱، نوع حمله را تشخیص دهد. داده attack-side ذاتاً mixed است؛ بنابراین وجود PCAP حمله به معنی حمله بودن همه پکتها نیست.

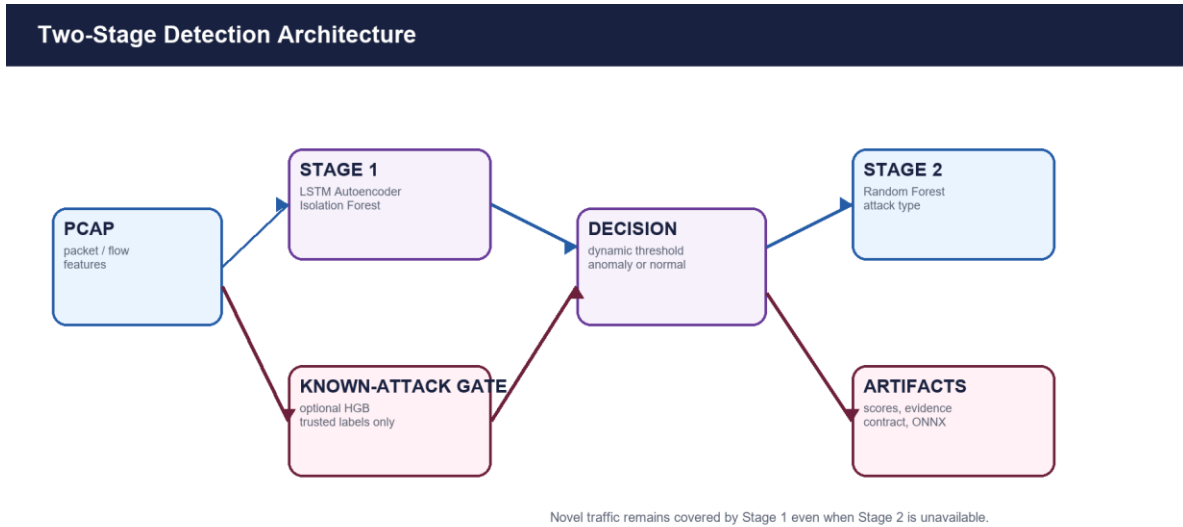
اهداف قابل سنجش

- حفظ FPR پایین روی capture نرمال دیده نشده، بدون تنظیم threshold با test data.
- افزایش recall حمله‌های دارای label معتبر، بدون برجسب گذاری جعلی پکت‌های unknown.
- عملکرد CPU-first، ورودی پویا بر اساس profile و خروجی قابل حمل در native و ONNX.
- ارائه evidence برای هر alert، نه صرفاً یک برجسب anomaly.

تصمیم	دلیل فنی
معماری دومرحله‌ای	کاهش هزینه classifier چندکلاسه روی همه ترافیک و جداسازی کشف از attribution.
PCAP-first	جلوگیری از نشت ستون CSV به featureهای مدل و حفظ قابلیت deployment روی ترافیک خام.
profile immutable	یکسان ماندن feature order و transformation میان train و inference.

۲. نمای معماری مدل دومرحله‌ای

ورودی Stage ۱ ماتریس transform شده از profile فریز شده است. این مرحله دو signal مکمل می‌سازد: reconstruction error و isolation score. در داده label دار کافی، گیت سبک supervised برای حمله‌های شناخته شده نیز اضافه می‌شود. Stage ۲ فقط برای anomaly های عبور کرده اجرا می‌شود.



شکل ۱ — معماری واقعی مدل و artifact های خروجی

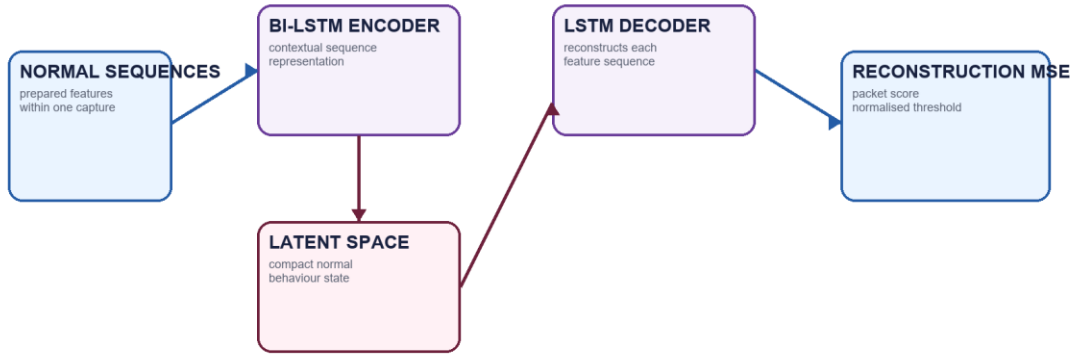
بخش	ورودی	خروجی
Stage ۱	capture group و feature matrix	score نرمال شده، flag، anomaly vote
گیت اختیاری	normal train + attack label دار از جدا capture	known-attack probability
Stage ۲	anomaly های عبور کرده	probability و attack type
Evidence	feature matrix + baseline normal	رتبه feature های دور از رفتار normal

۳. مدل LSTM Autoencoder

چرا LSTM-AE انتخاب شده است؟

رفتار شبکه فقط مقدار مستقل یک feature نیست؛ نرخ، فاصله زمانی، طول پکت، تکرار درخواست و رفتار flow در یک توالی معنا پیدا می‌کنند. LSTM-AE روی توالی normal آموزش می‌بیند و وقتی الگوی جدید قابل reconstruction نباشد، MSE بالاتری تولید می‌کند.

LSTM Autoencoder: Boundary-Safe Reconstruction



A sequence never crosses a PCAP capture boundary; short capture tails are padded deterministically.

شکل ۲ — encoder/latent/decoder و محاسبه reconstruction MSE

جزء	پایه‌سازی	دلیل
Encoder	Bi-LSTM + LayerNorm	استفاده از context دوطرفه در window برای نمایش پایدار normal.
Latent	linear + GELU + LayerNorm	فشرده‌سازی سبک و جلوگیری از latent بسیار بزرگ.
Decoder	LSTM و لایه خروجی feature	بازسازی توالی در همان فضای transform شده.
Dynamic architecture	اندازه براساس تعداد feature	profile کوچک مدل سبک‌تر و profile بزرگ بدون hard-code.
Capture boundary	sequence_groups	جلوگیری از ساخت sequence مصنوعی میان دو PCAP نامرتبط.

۴. Isolation Forest و ترکیب سیگنال‌ها

چرا Isolation Forest؟

Isolation Forest بدون نیاز به label، نمونه‌های دور از تراکم normal را با مسیرهای کوتاه‌تر در درخت‌ها جدا می‌کند. نسبت به Autoencoder یک failure mode متفاوت دارد: ممکن است پکت با MSE کم هنوز در فضای featureهای normal به‌آسانی isolate شود.

مدل روی همان prepared normal matrix fit می‌شود. score آن معکوس می‌گردد تا در تمام لایه‌ها مقدار بزرگ‌تر به معنی ناهنجاری بیشتر باشد. تعداد estimator و worker از config می‌آیند و CPU workerها محدود می‌شوند.

Ensemble و آستانه

scoreهای LSTM و Isolation Forest واحد یکسان ندارند. هر score ابتدا بر threshold calibration خودش تقسیم می‌شود. سپس `calibrated\_max` یا وزن‌های تعریف‌شده در config یک score بدون بُعد می‌سازند. threshold نهایی با distribution نرمال calibration می‌شود، نه با test attack.

حالت	کاربرد
calibrated_max	حفظ signal قوی هر detector؛ مناسب وقتی یک detector به یک حمله حساس‌تر است.
calibrated_weighted	ترکیب نرم وزن‌دار و bonus توافق detectorها.
both_detectors	حالت محافظه‌کار برای محیطی که FPR بسیار حساس است.
either_detector	سازگاری با قراردادهای قدیمی؛ حساس‌تر اما ممکن است FPR را بالا ببرد.

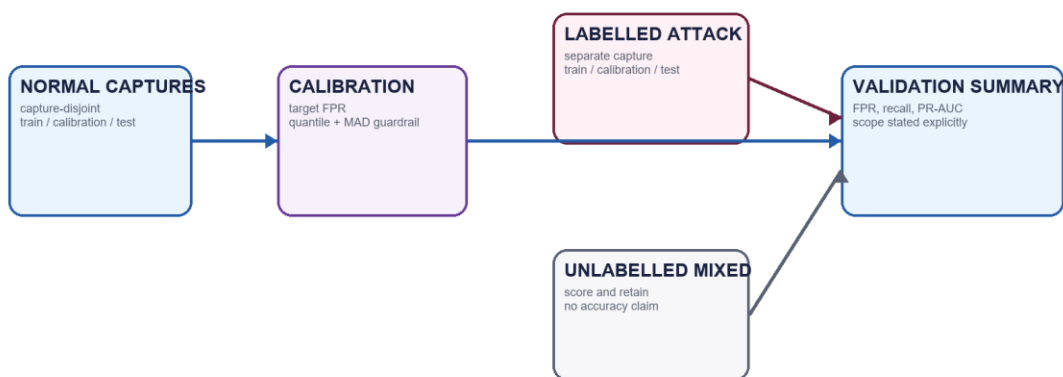
۵. گیت حمله شناسخته شده و کنترل false negative

دلیل افزودن گیت سبک

Autoencoder می‌تواند بعضی حمله‌ها را ساده‌تر از traffic normal متنوع بازسازی کند. در این وضعیت پایین آوردن threshold فقط false positive را زیاد می‌کند. گیت HistGradientBoosting یک classifier سبک CPU است که فقط از packet‌های واقعاً map و label شده استفاده می‌کند و در کنار novelty detector قرار می‌گیرد.

گیت تنها وقتی فعال می‌شود که حداقل سه capture دارای attack label معتبر وجود داشته باشد. capture‌های مثبت به train، calibration و evaluation تقسیم می‌شوند؛ بنابراین یک capture واحد نمی‌تواند هم یادگیری و هم موفقیت ظاهری مدل را بسازد.

Stage 1 Validation and Threshold Policy



Only trusted CSV-mapped packets contribute to accuracy metrics. Unknown labels are observations, never fabricated negatives.

شکل ۳ — سیاست label، calibration و تفکیک metric از observation

نوع رکورد	رفتار مدل/metric
normal PCAP	train، calibration و Stage ۱ holdout FPR.
trusted mapped attack	ورود به recall/F۱ و در صورت کافی بودن capture آموزش گیت.
attack capture در trusted mapped benign	نمونه منفی معتبر برای بررسی FPR distribution deployment.
unknown یا mapping ردشده	score و ذخیره برای review؛ حذف از TP/TN/FP/FN.

۶. Random Forest مرحله دوم

نقش و دلیل انتخاب

پس از گیت ۱ Stage، فقط تعداد کمتری candidate به Stage ۲ می‌رسند. Random Forest برای tabular featureهای ترکیبی، روابط غیرخطی، مقیاس‌پذیری CPU، پایداری نسبت به scaling و ارائه feature importance انتخاب شده است. این مدل به‌جای anomaly detection، attack-type classification انجام می‌دهد.

Stage ۲ matrix شامل transformed featureها و در صورت فعال‌بودن، scoreهای LSTM/Isolation Forest است. RF با `balanced\_subsample` آموزش می‌بیند تا classهای کم‌تعداد کاملاً نادیده گرفته نشوند.

دلیل	Readiness
جلوگیری از ساخت RF ظاهراً موفق با نمونه بسیار کم.	حداقل تعداد رکورد
تشخیص نوع حمله تک‌کلاسه ارزش classifier ندارد.	حداقل دو attack type
امکان split و metric معنادار.	حداقل نمونه در هر کلاس
جلوگیری از نشت پکت/flow یک capture در train و test.	capture-held-out split
در نبود label کافی، ۱ Stage همچنان deployable است.	skip امن

۷. ارزیابی، threshold و معیارهای اعتبار

روش تست درست

ابتدا FPR روی normal capture های دستنخورده بررسی می‌شود. سپس ROC-AUC، F1، precision، recall و Average Precision فقط روی packet‌هایی محاسبه می‌شوند که label CSV آن‌ها در mapping پذیرفته شده است. همه score ها و role در `stage1/scores.parquet` قابل audit هستند.

حمله‌ای که normal تشخیص داده شود False Negative است؛ normal ی که attack اعلام شود False Positive است. این دو هزینه عملیاتی متفاوت دارند و نباید تنها accuracy کلی گزارش شود.

معیار	سؤال عملیاتی
FPR / Specificity	چند درصد ترافیک normal باعث alert بی‌مورد می‌شود؟
Recall / FNR	چند درصد حمله labelدار از Stage ۱ عبور نمی‌کند؟
Precision	از alert‌ها چند مورد واقعاً attack هستند؟
PR-AUC	با عدم‌توازن شدید، رتبه‌بندی حمله‌ها چقدر مفید است؟
Latency / records/sec	مدل روی CPU server چه ظرفیت عملیاتی دارد؟
Weighted F1 Stage ۲	نوع حمله در تمام class‌ها چقدر درست تشخیص داده می‌شود؟

۸. explainability، خروجی و قرارداد inference

دلیل قابل توضیح بودن alert

خروجی ۱ Stage فقط boolean نیست. برای هر packet score LSTM، score Isolation Forest، vote، detector، threshold، score نهایی، probability گیت و `feature-evidence` ثبت می‌شود. evidence از انحراف robust feature نسبت به median/IQR normal در همان فضای transform شده ساخته می‌شود.

`model-contract.json` تنها مرجع inference است: مسیر profile، preprocessor، ترتیب ستون‌ها، native model، thresholdها و وضعیت Stage ۲ را نگه می‌دارد. `score\_two\_stage` ابتدا contract را می‌خواند و سپس همان preprocessing را اجرا می‌کند؛ از feature order جدید یا dashboard state استفاده نمی‌کند.

مصرف بیرونی	artifact
ورودی سرویس inference یا validator deployment.	model-contract.json
alert investigation و dashboard/API table.	stage۱/scores.parquet
توضیح top featureهای هر alert.	feature-evidence.parquet
خروجی نهایی normal/anomaly/type.	pipeline/end-to-end-predictions.parquet
runtimeهای ONNX سازگار، با حفظ contract preprocessing.	onnx/*.onnx

۹. منابع، بسته‌بندی و تصمیم‌های عملیاتی

CPU-first و کنترل منابع

مدل‌ها برای CPU طراحی شده‌اند. LSTM اندازه hidden/latent را از تعداد feature تعیین می‌کند، `max\_train\_windows` هزینه train را سقف می‌گذارد، Isolation Forest و RF worker کنترل‌شده دارند و diagnosticهای پرهزینه با estimator محدود اجرا می‌شوند. resource snapshot قبل/بعد train ذخیره می‌شود.

بسته‌بندی و deployment

هر model run native artifact و ONNX مستقل دارد. ONNX exporter نتیجه هر جزء را در `onnx-export.json` ثبت می‌کند تا شکست یک converter باعث توقف deployment native نشود. preprocessor joblib و manifest بخشی از مدل هستند، نه فایل فرعی اختیاری.

محدودیت‌ها و تصمیم بعدی

- بدون captureهای attack با label معتبر کافی، گیت و Stage ۲ عمداً train نمی‌شوند.
- اگر recall پایین و FPR پایین باشد، ابتدا mapping coverage و profile باید بررسی شود؛ پایین‌آوردن کورکورانه threshold راه حل نیست.
- برای Modbus یا protocol کم‌برچسب، افزایش captureهای label‌دار و بررسی نمونه‌گیری label-aware اولویت دارد.
- نتیجه smoke با ۲۰۰ پکت فقط صحت خروجی را می‌سنجد؛ معیار نهایی از full run و capture-held-out گزارش می‌شود.